DEVELOPER WORKFLOW

Jessica's Screen Editing & PR Collaboration Flow

How Jessica edits WRX-005 screens and pushes changes to production

For: Eddie Mitchell, Ray, Argos

From: Jessica Mitchell + Philo

Date: April 7, 2026

Purpose: Establish a collaboration workflow where Jessica edits screens locally, previews with live data, and submits changes via pull request

TL;DR

Jessica has a local development setup that serves Eddie's screen files with **live API data from the staging server**. She can edit screens, preview changes instantly, and submit them via **GitHub pull request** to Eddie's repo. Eddie/Ray/Argos review and merge. No risk to production—everything goes through a PR gate.

1. Repository Architecture

```
Eddie's Repo (upstream)
  github.com/eemism68/WhiteLaneWrx
  Branch: master · The source of truth
      ↓ nightly sync (4 AM) + manual pulls

Jessica's Fork (origin)
  github.com/jessicamitchell146/WhiteLaneWrx
  Branch: master · Jessica's safe copy
      ↓ git pull / git push

Jessica's Laptop (local)
  ~/MD Files VS Code/WhiteLaneWrx/
  Edits happen here · Proxy previews here
```

Jessica's fork stays in sync with Eddie's repo via an automated nightly sync at 4:00 AM. When Jessica makes edits, they flow **upstream** through a pull request—never directly to Eddie's repo.

2. Local Preview Environment

Jessica runs a **local proxy server** (built by Philo) that does two things:

REQUEST	HANDLED BY	SOURCE
Screen files `/screens/*.html`	Local filesystem	`~/MD Files VS Code/WhiteLaneWrx/app/wrx-005/screens/`
CSS, JS, assets	Local filesystem	`~/MD Files VS Code/WhiteLaneWrx/app/wrx-005/`
API calls `/api/*`	Staging server	`ssda.whitelanewrx.com` (live Supabase data)

This means Jessica sees her **local screen edits** rendered with **real Keiper project data**—exactly how they'll look in production.

```
# Start the proxy
node tools/screen-proxy/server.cjs

# Open in browser
http://localhost:3001/
# Shows index with links to all 4 screens
```

3. Edit → Preview → PR Workflow

1. **Edit a screen file locally**

   Jessica (with Philo's help) modifies files in `app/wrx-005/screens/`. These are self-contained HTML files with embedded CSS and JS.

2. **Preview with live data**

   Refresh `localhost:3001` in the browser. The proxy serves the edited file locally while API calls hit the staging server. Instant feedback with real data.

3. **Iterate until happy**

Edit → refresh → review. Repeat until the changes look right. Jessica can make design tweaks, add features, fix layouts—anything in the HTML/CSS/JS.

4

Create a feature branch

Before committing, Jessica creates a branch describing the change.

```
git checkout -b jessica/selection-tracker-color-codes
```

5

Commit and push to her fork

```
git add app/wrx-005/screens/selection-tracker-live.html
git commit -m "Add color-coded room completeness to selection tracker"
git push origin jessica/selection-tracker-color-codes
```

6

Open a Pull Request to Eddie's repo

Jessica creates a PR from her fork to `eemism68/WhiteLaneWrx:master`. The PR includes a summary of what changed and why, plus screenshots.

```
gh pr create --repo eemism68/WhiteLaneWrx \
  --title "Add color-coded room completeness" \
  --body "Based on Kevin's feedback from the 4/6 field visit..."
```

7

Eddie / Ray / Argos review and merge

Eddie reviews the diff, leaves comments if needed. Once approved, he merges. The changes are now in the main repo and will deploy to staging.

4. Safety Guarantees

CONCERN	HOW IT'S HANDLED
Jessica accidentally breaks a screen	Changes are on **her fork only** until a PR is merged. Production is untouched.
Jessica edits conflict with Eddie's work	Git detects merge conflicts in the PR. They resolve together before merging.
Jessica modifies backend / API code	She only edits screen files (HTML/CSS/JS). API and backend are on the staging server—read-only for her.
Nightly sync overwrites Jessica's changes	Sync only pulls **upstream → local**. Jessica's uncommitted work stays. Feature branches are isolated.
Jessica pushes directly to Eddie's repo	She **cannot**—she doesn't have write access to upstream. Everything goes through PRs.

5. What Jessica Can Edit

FILE TYPE	LOCATION	CAN EDIT?	EXAMPLES
Screen HTML files	`app/wrx-005/screens/`	YES	Layout, colors, navigation, labels, new UI features
CSS tokens / styles	`app/wrx-005/`	YES	Design system tokens, color palette, typography
Docs & specs	`wrx-005/docs/`	YES	Feature specs, meeting notes, analysis docs
Backend / API routes	`wrx-005/services/`	WITH GUIDANCE	Philo assists; Eddie/Argos review in PR
Database / migrations	`wrx-005/docs/*.sql`	READ ONLY	Jessica reads; Eddie/Argos modify

6. Notes for Ray & Argos

When Jessica Opens a PR

- PRs from `jessicamitchell146/WhiteLaneWrx` are Jessica's work, assisted by Philo
- Changes will primarily be in `app/wrx-005/screens/*.html` (UI/UX edits)
- PRs will include context from field visits and user feedback (Kevin, Travis, Scott)
- Jessica's branch names will follow: `jessica/<feature-description>`
- Co-authored commits may include: `Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>`

Review Expectations

- Jessica's edits are **design and UX driven**—informed by real field user feedback
- She has strong taste (colors, density, layout, functional beauty)—trust her instincts on visual design
- Philo handles code quality; Jessica handles user experience
- If a screen change requires a backend/API change, Jessica will note it in the PR description for Argos to implement

Collaboration Model

Jessica + Philo *(UX, design, field feedback)*

↓ *Pull Request with screenshots + context*

Eddie / Ray *(review, approve, strategic direction)*

↓ *Merge + deploy*

Argos *(backend changes if needed, flagged in PR)*

↓ *Implements API/DB changes in companion commit*

Key Principle: Jessica owns the customer experience. Her edits come directly from field visits, user interviews, and hands-on testing with Bespoke's team. When she says "Kevin needs this," it's because Kevin literally said it—on tape.

7. Current Screen Inventory

SCREEN	FILE	STATUS	LIVE PREVIEW
Mission Control (Command Center)	`command-center-live.html`	Live data	`localhost:3001/screens/command-center-live.html`
Floor Plan / Room View	`scope-rooms.html`	Live data	`localhost:3001/screens/scope-rooms.html`
Living Scope Explorer	`scope-explorer-live.html`	Live data	`localhost:3001/screens/scope-explorer-live.html`
Selection Tracker	`selection-tracker-live.html`	Live data	`localhost:3001/screens/selection-tracker-live.html`

White Lane AI / BuilderWrx

Jessica's Development Workflow — April 7, 2026

Prepared by Philo